

Chrysler Scanner V2 API Reference

Last Updated: June 5, 2026

1. Overview

The Chrysler Scanner firmware acts as a high-speed bridge between modern host applications (Windows, macOS, Android, iOS) and legacy Chrysler automotive networks (CCD, PCI, and SCI).

Communication with the scanner is purely asynchronous, packet-based, and identical across both USB and Bluetooth LE (BLE) transport layers. The scanner operates in a Master/Slave relationship with the host application: the host issues requests or commands, and the scanner responds. The scanner will also independently push asynchronous events (like incoming automotive bus messages) to the host.

2. Transport Layers

2.1 USB (Serial Port)

- **Baud Rate:** 250000 bps (Default, configurable via API)
- **Data Bits:** 8
- **Parity:** None
- **Stop Bits:** 1
- **Flow Control:** None

2.2 Bluetooth Low Energy (BLE)

The scanner implements the standard **Nordic UART Service (NUS)** to emulate a serial stream over BLE.

- **Service UUID:** 6E400001-B5A3-F393-E0A9-E50E24DCCA9E
- **RX Characteristic (Host -> Scanner):** 6E400002-B5A3-F393-E0A9-E50E24DCCA9E (Write / Write Without Response)
- **TX Characteristic (Scanner -> Host):** 6E400003-B5A3-F393-E0A9-E50E24DCCA9E (Notify)

Note: BLE MTU size negotiation is supported and highly recommended to maximize throughput.

3. Packet Framing Protocol

Every byte of data exchanged between the host and the scanner is wrapped in a strict packet frame. The framing protocol ensures data integrity, synchronizes streams, and explicitly routes payloads.

3.1 Packet Structure

Byte Offset	Field Name	Size	Type	Description
0	Sync	1 byte	uint8	Always 0x3D (=). Used to synchronize the stream.
1 - 2	Length	2 bytes	uint16	Total length of the Payload + 2 (Big Endian).
3	Routing	1 byte	uint8	Bitmapped field containing Direction, Bus ID, and Command.
4	Mode	1 byte	uint8	Sub-command modifier (varies by Command).
5 to N-2	Payload	N bytes	uint8[]	Optional data. Length = Length field - 2.
N-1	Checksum	1 byte	uint8	8-bit additive sum of all preceding bytes.

Total Packet Size Calculation:

Total Size = Payload Length + 6 bytes

3.2 The Routing Byte (Byte 3)

The routing byte is a bitmapped flag that determines where the packet came from, what subsystem it targets, and what action to perform.

Bit 7 (MSB)	Bit 6-4	Bit 3-0 (LSB)
Direction	Bus ID	Command ID

- **Direction Bit:**

- 0 = Host to Scanner (Requests)
- 1 = Scanner to Host (Responses)

- **Bus ID (Bits 6-4):**

- 0x0 = USB / Internal System
- 0x1 = CCD Bus
- 0x2 = SCI Bus (PCM / Engine)
- 0x3 = SCI Bus (TCM / Transmission)
- 0x4 = PCI Bus

- **Command ID (Bits 3-0):**

- 0x0 = Reset
- 0x1 = Handshake
- 0x2 = Status
- 0x3 = Settings
- 0x4 = Request
- 0x5 = Response
- 0x6 = Message TX (Send to Car)
- 0x7 = Message RX (Received from Car)
- 0xC = File Operations
- 0xD = Bootloader / ECU Flashing
- 0xE = Debug / Emulator
- 0xF = Error

3.3 Checksum Calculation

The checksum is a simple 8-bit unsigned rollover sum of all bytes in the packet *excluding* the checksum byte itself.

C# / C++ Example:

```

public byte CalculateChecksum(byte[] buffer, int lengthWithoutChecksum)
{
    byte checksum = 0;
    for (int i = 0; i < lengthWithoutChecksum; i++)
    {
        checksum += buffer[i]; // Rollover is intentional
    }
    return checksum;
}

```

4. Packet Construction Example

Scenario: Host wants to request the scanner's Hardware/Firmware Info.

- Sync : 0x3D
- Length : 0x0002 (0 Payload bytes + 2)
- Routing : 0x04 (Direction=0, Bus=USB[0], Cmd=Request[4])
- Mode : 0x01 (Req_HwFw_Info)
- Payload : [None]

Raw Host TX: 3D 00 02 04 01 44 (*0x44 is the sum of 3D+00+02+04+01*)

Scanner Response:

- Sync : 0x3D
- Length : 0x0019 (23 Payload bytes + 2)
- Routing : 0x85 (Direction=1, Bus=USB[0], Cmd=Response[5])
- Mode : 0x01 (Res_HwFw_Info)
- Payload : [23 bytes of Hardware/Firmware Data]

5. Global Error Responses

If the host sends an invalid packet or the scanner encounters an internal error, it will respond with an **Error Packet**.

Routing Byte: `0x8F` (Scanner to Host, USB Bus, Error Command) **Payload:** Usually a single `0xFF` byte, though it can be ignored. **Mode Byte (The Error Code):**

- `0x00` = OK (Acknowledge)
- `0x01` = Length Invalid Value
- `0x02` = Invalid Command (Command ID not recognized)
- `0x03` = Invalid Mode (Mode/Sub-command not recognized)
- `0x04` = Invalid Payload Values
- `0x05` = Checksum Invalid
- `0x06` = Packet Timeout (RX stream stalled)
- `0x07` = Buffer Overflow
- `0x08` = Invalid Bus
- `0xFE` = Internal Execution Error
- `0xFF` = Fatal Error

Example of a standard "OK" ACK from the scanner (Length=3, Payload=[0x00]):

`3D 00 03 8F 00 00 CF`

6. Core System Commands

These commands handle the fundamental lifecycle and identification of the scanner.

6.1 Reset Scanner (`Command ID: 0x00`)

Reboots the ESP32 microcontroller or notifies the host that a reboot has occurred.

- **Host Request Mode:** `0x00` (`rm_init`)
 - **Payload:** 1 byte (Reset reason requested by host, user-defined)
- **Scanner Response Mode:** `0x01` (`rm_done`)
 - **Payload:** 1 byte (Actual hardware reset reason, via `esp_reset_reason()`)
 - *Note: The scanner broadcasts this automatically upon startup.*

6.2 Handshake (`Command ID: 0x01`)

Used to identify the device and verify communication channels are active.

- **Host Request Mode:** `0x00` (`hm_none`)

- **Payload:** None
- **Scanner Response Mode:** 0x00
 - **Payload:** 15 bytes (ASCII String: CHRYSLERSCANNER)

6.3 Status (Command ID: 0x02)

Retrieves a comprehensive real-time snapshot of the scanner's internal state, memory, voltages, and bus statistics.

- **Host Request Mode:** 0x00 (sm_none)
 - **Payload:** None
- **Scanner Response Mode:** 0x00
 - **Payload:** 45 bytes (All multi-byte integers are **Big-Endian**)

Byte Offset	Type	Description
0 - 3	uint32	Device Uptime in milliseconds
4 - 7	uint32	RAM Usage (Total Heap - Free Heap) in bytes
8 - 9	uint16	Battery Voltage (mV)
10 - 11	uint16	Bootstrap Voltage / VBB (mV)
12 - 13	uint16	Programming Voltage / VPP (mV)
14	uint8	Current CCD-Bus Settings Byte
15 - 18	uint32	CCD-Bus RX Packet Count
19 - 22	uint32	CCD-Bus TX Packet Count
23	uint8	Current PCI-Bus Settings Byte
24 - 27	uint32	PCI-Bus RX Packet Count
28 - 31	uint32	PCI-Bus TX Packet Count
32	uint8	Current SCI-Bus Settings Byte
33 - 36	uint32	SCI-Bus RX Packet Count
37 - 40	uint32	SCI-Bus TX Packet Count
41 - 42	uint16	LED Heartbeat Interval (ms)
43 - 44	uint16	LED Blink Duration (ms)

7. Device Settings (`Command ID: 0x03`)

Settings commands configure hardware peripherals. The scanner automatically saves these parameters to Non-Volatile Storage (NVS) and acknowledges by echoing the packet back (`Direction` bit set to `1`).

7.1 Set LED Mode (`Mode: 0x01`)

Configures the device's status LEDs.

- **Payload:** 4 bytes

- Bytes 0-1: Heartbeat Interval (ms) (`0` = disable heartbeat)
- Bytes 2-3: Blink Duration (ms) (`0` = completely disable LEDs)

7.2 Set CCD-Bus Configuration (`Mode: 0x02`)

- **Payload:** 1 byte (Bit 7: `1` = Enable Bus, `0` = Disable Bus)

7.3 Set PCI-Bus Configuration (`Mode: 0x06`)

- **Payload:** 1 byte (Bit 7: `1` = Enable Bus, `0` = Disable Bus)

7.4 Set SCI-Bus Configuration (`Mode: 0x03`)

SCI setup requires strict multiplexing of the `AW9523` port expander and UART logic levels.

- **Payload:** 1 byte (Bitmapped)

Bit(s)	Function	Description
<code>7</code>	Enable	<code>1</code> = Enable transceiver, <code>0</code> = Disable
<code>6</code>	Nibble Swap	<code>1</code> = Swap 4-bit nibbles in received/transmitted bytes
<code>5</code>	Module Target	<code>0</code> = PCM (Engine), <code>1</code> = TCM (Transmission)
<code>4</code>	NGC Mode	<code>1</code> = Next Generation Controller, <code>0</code> = Legacy (SBEC/JTEC/EATX)
<code>3</code>	Logic Polarity	<code>1</code> = Inverted / OBD1, <code>0</code> = Normal / OBD2
<code>2</code>	OBD Config	Pin routing switch. <code>0</code> = Config A, <code>1</code> = Config B
<code>1 - 0</code>	Baudrate	<code>0</code> = 976, <code>1</code> = 7812, <code>2</code> = 62500, <code>3</code> = 125000 bps

7.5 Apply SCI VXX Strobe Voltage (`Mode: 0x07`)

Activates hardware level-shifters to apply high voltage directly to the active SCI-TX pin. Crucial for flashing or unlocking bootstrap modes.

- **Payload:** 2 bytes (`uint16` , Big-Endian) in millivolts.
 - `0x0000` (`0`) = `VXX_NONE` (Off)
 - `0x2EE0` (`12000`) = `VBB` (Bootstrap Voltage)

- `0x4E20` (20000) = `VPP` (Programming Voltage)

7.6 Set USB UART Baudrate (`Mode: 0x08`)

Changes the baud rate of the primary USB Serial connection.

- **Payload:** 4 bytes (`uint32` , Big-Endian). *Default is 250,000.*

8. System Requests (`Command ID: 0x04`)

Requests prompt the scanner to return specific diagnostic information. The scanner responds with `Command ID: 0x05` .

Request Mode (<code>0x04</code>)	Response Mode (<code>0x05</code>)	Payload Description (Response)
<code>0x01</code> (HW/ FW Info)	<code>0x01</code>	23 Bytes: HW Ver (4 bytes, nibble-split), FW Major, Minor, Build, Rev (4 bytes), Chip Model (1 byte), Chip Rev (2 bytes), Cores (1 byte), Features (4 bytes), Flash Size in MB (1 byte), Bluetooth MAC Address (6 bytes).
<code>0x02</code> (Timestamp)	<code>0x02</code>	4 Bytes: <code>uint32</code> Milliseconds since boot.
<code>0x03</code> (Battery Volts)	<code>0x03</code>	2 Bytes: <code>uint16</code> Voltage in mV.
<code>0x06</code> (VBB Volts)	<code>0x06</code>	2 Bytes: <code>uint16</code> 12V Bootstrap output in mV.
<code>0x07</code> (VPP Volts)	<code>0x07</code>	2 Bytes: <code>uint16</code> 20V Programming output in mV.
<code>0x08</code> (All Voltages)	<code>0x08</code>	6 Bytes: Batt, VBB, VPP (<code>uint16</code> x 3).

9. Debug & OTA Updates (`Command ID: 0x0E`)

These sub-commands interact with the ESP32's deeper system functions, including Over-The-Air firmware updates and the internal hardware SCI emulator.

9.1 SCI Emulator Commands

The scanner can physically simulate a Chrysler PCM/TCM on the SCI bus for software testing without a real vehicle.

- **Enable/Disable Emulator (Mode: 0x0B):**
 - Payload: 0x01 (Enable), 0x00 (Disable)
- **Set Emulator OBD1 Mode (Mode: 0x0D):**
 - Payload: 0x01 (OBD1 Logic), 0x00 (OBD2 Logic)
- **Set Emulator ECM Mode (Mode: 0x0E):**
 - Payload: 0x01 (Cummins Diesel Engine Controller), 0x00 (Normal Engine Controller)
- **Upload Emulator Data Blob (Mode: 0x0C):**
 - Payload: Up to 1024 bytes. This binary blob defines the simulated diagnostic fault codes, ROM IDs, and sensor responses. The scanner stores this blob in NVS. Blob format is not public yet.

9.2 Over-The-Air (OTA) Firmware Updates

Allows 3rd-party apps to flash new ESP32 firmware updates directly to the scanner over USB or BLE.

1. **Start OTA (Mode: 0x15):** Payload: None. Opens the inactive OTA partition.
2. **Set Hash (Mode: 0x17):** Payload: 32 bytes (SHA-256 Hash of the .bin file). *Highly recommended to verify integrity.*
3. **Write Chunk (Mode: 0x18):** Payload: Variable length data chunk. Send sequentially until the .bin file is fully transferred.
4. **Finalize OTA (Mode: 0x19):** Payload: None. Verifies the SHA-256 hash against the written partition, sets the boot flag, and automatically reboots the scanner.
5. **Abort OTA (Mode: 0x16):** Payload: None. Cancels an active OTA and cleans up.
6. **Revert Firmware (Mode: 0x1A):** Payload: None. Rolls the scanner back to the previously installed firmware version and reboots.

9.3 System Debug

- **Get Task Stack Statistics (Mode: 0x20):** Payload: None. Prints FreeRTOS High-Water Marks to the debug UART to check for memory leaks.
- **Set BLE MTU Size (Mode: 0x30):**

- Payload: 2 bytes (`uint16` Big-Endian). Requests a larger Maximum Transmission Unit for BLE before connection to maximize Bluetooth bandwidth.

10. Automotive Bus Messaging

The scanner allows raw data injection and extraction from the vehicle's CCD, PCI, and SCI networks.

- **Host to Scanner (Transmit):** Command ID `0x06` (`cmd_msg_tx`)
- **Scanner to Host (Receive):** Command ID `0x07` (`cmd_msg_rx`)

10.1 Receiving Messages from the Vehicle

When the scanner receives a valid message on an active bus, it wraps it in an `0x07` packet and pushes it asynchronously to the host.

Response Mode Byte:

- `0x01` : CCD, PCI, or SCI Low-Speed message.
- `0x02` : SCI High-Speed message (Multiplexed/Streamed/Bootstrap).

Payload Structure (RX):

Byte Offset	Type	Description
<code>0 - 3</code>	<code>uint32</code>	Timestamp: Milliseconds since the scanner booted (Big-Endian). Extremely useful for Host-side charting/logging.
<code>4 - N</code>	<code>uint8[]</code>	Raw Bus Message: The actual bytes captured from the vehicle network (including checksums/CRCs if applicable).

10.2 Transmitting Messages to the Vehicle

When transmitting, the host specifies *how* the scanner should push the data using the **Mode** byte.

- `0x01` (**Stop**): Stops any active repeating transmission loops. (Payload: None)
- `0x02` (**Single**): Transmits the payload once.
- `0x03` (**List**): Transmits a sequence of different messages sequentially.

- **0x04 (Repeat):** Transmits a message (or sequence) infinitely at a strict hardware-timed interval.
- **0x81 (Single + VBB):** *[SCI Bus Only]* Transmits the payload once, then immediately applies 12V Bootstrap Voltage to the SCI-TX pin (used for waking up PCMs in Bootstrap mode).
- **0x82 (Single + VPP):** *[SCI Bus Only]* Transmits the payload once, then immediately applies 20V Programming Voltage to the SCI-TX pin (used for unlocking Flash memory).

Payload Structure (Single TX 0x02 , 0x81 , 0x82): Simply append the raw bytes. *Note: For CCD, the scanner hardware automatically calculates and appends the 8-bit checksum. For PCI, it calculates and appends the J1850 CRC. For SCI (Legacy), the host must provide the exact bytes. For SCI (NGC), the scanner appends the checksum automatically.*

Payload Structure (Repeat / List TX 0x03 , 0x04):

Byte Offset	Type	Description
0 - 1	uint16	Interval: Delay in milliseconds between transmissions.
2	uint8	Length (L1): Length of the first message.
3 - (3+L1-1)	uint8[]	Message 1 Data
3+L1	uint8	Length (L2): Length of the second message (optional).
...	...	... repeats up to 1024 bytes of buffer space.

11. Internal Storage (LittleFS)

The scanner uses an internal Flash partition (LittleFS) to store firmware for backup dumps or files intended for writing. Command ID: **0x0C**.

Mode (0x0C)	Action	Host Payload	Scanner Response Payload
0x01	Storage Info	None	Total Bytes (uint32), Used Bytes (uint32), followed by a concatenated list of files (see 11.1).
0x02	File Info	Null-terminated string (Filename)	Filename (\0), Exists (uint8, 1=true), Size (uint32).
0x03	Upload File	Filename (\0), followed by raw byte chunk.	Standard ACK (0x00) or Error (0xFF).
0x04	Download File	Filename (\0), Block Size (uint16).	Scanner starts pushing chunks to host via Mode 0x04 .
0x05	Rename File	Old Name (\0), New Name (\0), Overwrite (uint8 boolean).	Standard ACK or Error.
0x06	Delete File	Null-terminated string (Filename).	Standard ACK or Error.
0x07	Format	None. (Wipes the entire LittleFS partition).	Standard ACK or Error.
0x08	Cancel DL	None. Stops an active 0x04 download stream.	Standard ACK.
0x10	Read Chunk	Filename (\0), Offset (uint32), Length (uint32).	Scanner returns the exact byte chunk requested.

11.1 Storage Info File List Format

When requesting Storage Info (0x01), the file list appended to the payload is structured as follows: [Total Files (uint8)] [Record Length (uint8)] [Filename (\0)] [Size (uint32)] ... *Example:* 01 12 74 65 73 74 2E 62 69 6E 00 00 00 00 80 -> 1 File, 18 bytes record length, "test.bin", 128 bytes size.

12. ECU Bootloader & Flashing (Autoboot)

Flashing vintage engine controllers requires microsecond-perfect timing to inject bootstrap code into the target MCU's RAM. Because USB/BLE latency is unpredictable, the **ESP32 handles the entire state machine natively**. The Host app simply configures the job and monitors progress.

Command ID: `0x0D` (`cmd_boot`)

12.1 Target ECU Loaders (Byte 0)

- `0x01` = SBEC2
- `0x02` = SBEC3 / SBEC3+
- `0x03` = SBEC3A / SBEC3A+ / SBEC3B
- `0x04` = EATX3 / EATX3A (Transmission)
- `0x05` = EATX3B (Transmission)
- `0x06` = EATX4 (Transmission)
- `0x07` = JTEC
- `0x08` = JTEC+

12.2 Operation Modes

The Host sends an `0x0D` packet with one of the following **Modes**:

Automated State Machines

- `0x01` (`bm_flash_write`): Full Flash Write cycle (Unlock, Erase, Write, Verify).
- `0x02` (`bm_flash_read`): Full Flash Backup cycle (Unlock, Read to LittleFS).
- `0x03` (`bm_eeprom_write`): EEPROM Write cycle.
- `0x04` (`bm_eeprom_read`): EEPROM Read cycle.
- `0x0A` (`bm_read_68hc11`): Read Co-Processor memory.

Host Payload for Automated Modes:

Byte	Description
0	Loader ID (e.g., 0x01 for SBEC2)
1	Flags (Bitmapped) Bit 0: Backup original to LittleFS before writing (1 = yes) Bit 1: Is 64K Chip? (1 = 64K, 0 = 32K, SBEC2 only) Bit 2: Is Jeep variant? (1 = Jeep, SBEC2 only) Bit 3: Deep Erase (1 = Force full wipe)
2 - N	Null-Terminated String: Filename in LittleFS to Read from.

State Machine Control

- 0x09 (bm_cancellation_request): Safely aborts an active Autoboot sequence.
(Payload: None)
- 0x0B (bm_boot_next_task_request): Acknowledges a host prompt. *(Note: SBEC2 controllers must be power-cycled during the write sequence. The ESP32 will pause the state machine and wait for the Host to send 0x0B after the user cycles the ignition key).*

12.3 Status Notifications

While the Autoboot sequence runs, the scanner pushes continuous status updates to the Host (Direction = 1, Command = 0x0D).

The **Payload** of these status updates contains a 1-byte Status Code, optionally followed by data.

Success / Progression Codes:

- 0x00 : Job Complete Successfully.
- 128 : Verifying Voltages...
- 129 : Voltages OK.
- 132 : Part Number Extracted. (Payload: ASCII String of Part Number).
- 133 : Backup Created. (Payload: ASCII String of generated backup filename).
- 136 : Flash IDs Extracted. (Payload: 2 bytes [Manufacturer ID] [Chip ID]).
- 142 : Flash Erase Result. (Payload: 1 byte response from worker, 0x22 is success).
- 168 : Re-uploading Bootloader (Prompting user for Ignition Cycle).

Error Codes (Causes immediate sequence halt):

- 01 : Flash File Missing in LittleFS.
- 02 : Flash File Size Mismatch.
- 03 : Storage Full (Cannot create backup).
- 05 : Battery Voltage Low (< 11.5V).
- 06 : Bootstrap Voltage VBB Low (< 11.5V).
- 07 : Programming Voltage VPP Low (< 19.0V).
- 12 : Flash Checksum Mismatch (Write verification failed).
- 18 : Upload Worker Timeout.
- 19 : Start Worker Timeout.
- 22 : Task Cancelled by Host.